



Fundamentos de JavaScript

Comparaciones, lógica, decisiones, colecciones y ciclos

COMPARAR

===

Evaluar valores y obtener true o false.

DECIDIR

if

Ejecutar un camino según una condición.

REPETIR

for

Recorrer datos sin duplicar instrucciones.

JAVASCRIPT

LÓGICA

CONTROL DE FLUJO

ESTUDIANTE	BOOTCAMP	FECHA
Juan Andrés Suárez Gaviria	Bootcamp en Desarrollo Web Fullstack	28 de agosto de 2026

Actividad de investigación · Bootcamp en desarrollo web

Operadores de comparación

DEFINICIÓN

Los operadores de comparación enfrentan dos valores y producen un resultado booleano: true cuando la comparación se cumple o false cuando no se cumple. Se usan para formular preguntas dentro del programa, por ejemplo, si una edad es suficiente o si dos datos son iguales.

Sintaxis

valorA operador valorB

Tabla de operadores

Operador	Significado	Ejemplo y resultado
==	Igualdad con conversión de tipos.	5 == "5" → true
!=	Desigualdad con conversión de tipos.	5 != "6" → true
===	Igualdad estricta: compara valor y tipo.	5 === "5" → false
!==	Desigualdad estricta: valor o tipo diferente.	5 !== "5" → true
>	Mayor que.	10 > 4 → true
<	Menor que.	3 < 8 → true
>=	Mayor o igual que.	18 >= 18 → true
<=	Menor o igual que.	6 <= 5 → false

Ejemplos

EJEMPLO 1 · EDAD MÍNIMA

```
1 const edad = 20;
2 const puedeIngresar = edad >= 18;
3
4 console.log(puedeIngresar);
5 // Resultado: true
```

EJEMPLO 2 · IGUALDAD

```
1 const numero = 5;
2 const texto = "5";
3
4 console.log(numero == texto); // true
5 console.log(numero === texto); // false
```

Buena práctica: normalmente conviene usar === y !==, porque también verifican el tipo de dato y evitan conversiones inesperadas.

Operadores lógicos

DEFINICIÓN

Los operadores lógicos permiten combinar o negar condiciones. Trabajan con valores booleanos y ayudan a responder preguntas más completas, como: ¿el usuario tiene permiso y no está bloqueado?

Operadores principales

&&

AND
Devuelve true solo cuando las dos condiciones son verdaderas.

||

OR
Devuelve true cuando al menos una condición es verdadera.

!

NOT
Invierte el valor: true pasa a false y false pasa a true.

Sintaxis y resultados

Expresión	A = true, B = true	A = true, B = false	A = false, B = false
A && B	true	false	false
A B	true	true	false
!A	false	false	true

Ejemplos

EJEMPLO 1 · INICIO DE SESIÓN

```
1 const tieneCuenta = true;
2 const claveCorrecta = true;
3
4 const puedeEntrar =
5   tieneCuenta && claveCorrecta;
6 console.log(puedeEntrar); // true
```

EJEMPLO 2 · ACCESO AL CURSO

```
1 const esEstudiante = false;
2 const esDocente = true;
3 const estaBloqueado = false;
4
5 const tieneAcceso =
6   (esEstudiante || esDocente) &&
7   !estaBloqueado; // true
```

Los paréntesis ayudan a mostrar qué operación debe evaluarse primero y hacen que las condiciones combinadas sean más fáciles de leer.

Condicionales

DEFINICIÓN

Las estructuras condicionales permiten que el programa tome decisiones. JavaScript evalúa una condición y ejecuta solamente el bloque de instrucciones que corresponda al resultado.

if, else if y else

if

Evalúa la primera condición. Si es verdadera, ejecuta su bloque.

else if

Prueba otra condición cuando las anteriores fueron falsas.

else

Se ejecuta cuando ninguna de las condiciones anteriores se cumple.

Sintaxis

ESTRUCTURA COMPLETA

```
1 if (condicionPrincipal) {  
2   // Se ejecuta si la primera condición es verdadera  
3 } else if (otraCondicion) {  
4   // Se ejecuta si la segunda condición es verdadera  
5 } else {  
6   // Se ejecuta si ninguna condición se cumple  
7 }
```

Ejemplos

EJEMPLO 1 · MAYORÍA DE EDAD

```
1 const edad = 16;  
2  
3 if (edad >= 18) {  
4   console.log("Puede ingresar");  
5 } else {  
6   console.log("Debe ser mayor");  
7 }
```

EJEMPLO 2 · RESULTADO DE NOTA

```
1 const nota = 4.2;  
2  
3 if (nota >= 4.5) {  
4   console.log("Excelente");  
5 } else if (nota >= 3) {  
6   console.log("Aprobado");  
7 } else {  
8   console.log("No aprobado");  
9 }
```

Resultados: el primer ejemplo muestra "Debe ser mayor" y el segundo muestra "Aprobado".

Arrays

DEFINICIÓN

Un array es una estructura que guarda varios valores dentro de una sola variable. Sus elementos mantienen un orden y cada posición se identifica con un índice que comienza en 0.

Creación y acceso

```
const lenguajes = ["HTML", "CSS", "JavaScript"];
```

HTML

índice 0

CSS

índice 1

JavaScript

índice 2

`lenguajes[2]` → "JavaScript"

`lenguajes.length` → 3

Métodos básicos

Método	Qué hace	Ejemplo
<code>push()</code>	Agrega un elemento al final.	<code>datos.push("JS")</code>
<code>pop()</code>	Elimina y devuelve el último elemento.	<code>datos.pop()</code>
<code>unshift()</code>	Agrega un elemento al inicio.	<code>datos.unshift("HTML")</code>
<code>shift()</code>	Elimina y devuelve el primer elemento.	<code>datos.shift()</code>
<code>includes()</code>	Indica si un valor está en el array.	<code>datos.includes("CSS")</code>
<code>indexOf()</code>	Devuelve el índice de un valor o -1.	<code>datos.indexOf("JS")</code>

Ejemplos

EJEMPLO 1 · AGREGAR Y ACCEDER

```
1 const tareas = ["HTML", "CSS"];
2 tareas.push("JavaScript");
3
4 console.log(tareas[2]);
5 // Resultado: JavaScript
```

EJEMPLO 2 · ELIMINAR Y BUSCAR

```
1 const carrito = ["Mouse", "Teclado",
2   "Monitor"];
3 const ultimo = carrito.pop();
4 console.log(ultimo); // Monitor
5 console.log(carrito.includes("Mouse"));
6 // Resultado: true
```

Ciclos

DEFINICIÓN

Los ciclos repiten un bloque de instrucciones. Son útiles para recorrer arrays, contar, procesar datos o continuar una tarea mientras una condición siga siendo verdadera.

for y while

for

Se usa cuando conocemos el inicio, la condición y la actualización del contador.

```
for (inicio; condicion; cambio) {  
  // instrucciones  
}
```

while

Se usa cuando la repetición depende de una condición y no siempre sabemos cuántas vueltas habrá.

```
while (condicion) {  
  // instrucciones  
}
```

Ejemplos

EJEMPLO 1 · RECORRER CON FOR

```
1 const temas = ["HTML", "CSS", "JS"];  
2  
3 for (let i = 0; i < temas.length; i++) {  
4   console.log(`${i + 1}. ${temas[i]}`);  
5 }
```

SALIDA DEL EJEMPLO 1

1. HTML
2. CSS
3. JS

EJEMPLO 2 · CUENTA REGRESIVA

```
1 let contador = 3;  
2  
3 while (contador > 0) {  
4   console.log(contador);  
5   contador--;  
6 }  
7 console.log("¡Listo!");
```

SALIDA DEL EJEMPLO 2

```
3  
2  
1  
¡Listo!
```

Importante: en un **while** la condición debe cambiar dentro del ciclo; de lo contrario, puede producirse un ciclo infinito.